The Prompt Keeps Disappearing

Commands, Context, Programs, and the Trouble with Finding the Object

Watson Hartsoe

Working paper · 18 August 2026

AI-augmented working paper

Abstract

A prompt looks easy to locate: it is the thing the user typed. That confidence does not survive contact with systems that remember previous turns, resolve deictic references, expose tools, enforce schemas, generate temporary programs, inspect external state, and return outputs that users reject, repair, select, or rewrite. The visible utterance remains important, but its apparent agency repeatedly escapes into what surrounds it. Calling the prompt a command misses how reference can be built across an interaction. Calling it a description mistakes apparatus for semantics. Calling it a program ignores both the machinery that interprets it and the knowledge that programming practice has always carried beyond program text. Calling it a specification falters when failure and tacit judgment continue to determine what the task was. Calling the remainder "context" gathers mechanisms that do not behave alike. Provenance records production without deciding authorship; distribution names multiplicity without yet explaining transformation. The result is not a better definition. It is a recurring instability: the tighter the analytical boundary around the prompt, the more causal work appears outside it; the wider the boundary becomes, the less "the prompt" names a distinct object. This paper follows that instability rather than repairing it.

Keywords: prompting; natural-language programming; situated action; deixis; specification; authorship; distributed cognition; human-computer interaction

1 The thing in the box

A prompt seems to arrive already boxed. There is a text field. Someone types into it. The system answers. The interface gives us a noun almost for free: *the prompt*. It is tempting to let the visual boundary become the theoretical one.

That works until the user writes, "that one." Nothing in those two words identifies the object. The reference may have been prepared by the previous turn, by a highlighted region of an image, by a cursor position, by a remembered choice, or by a shared sequence of failed attempts. The utterance is visible. What makes it intelligible may not be.

The same trouble appears from the other side when the user writes a long, explicit instruction. A sentence can look self-sufficient and still owe its effect to machinery it does not contain: tool descriptions, permissions, schemas, runtime state, model parameters, hidden instructions, external files. The words can be held constant while the result changes because the world around the words changed.

This is not an exotic edge case. It recurs whenever prompting becomes interesting. Every time the prompt is isolated tightly enough to explain what it does, some of its apparent agency leaks into something outside the cut. Enlarge the cut to catch the leak and the object starts to dissolve. Soon "the prompt" means the utterance, the dialogue, the tool environment, the model, the task history, and the user's later judgment. A category that includes everything can no longer tell us what did what.

The problem is therefore not simply that prompting is complicated. Writing was already complicated. Programming was already complicated. The stranger problem is that the object presented by the interface and the object needed by the explanation keep drifting apart.

2 A command that points elsewhere

The command is the cleanest picture. A user issues an instruction; a system follows it. Contemporary agents make the picture concrete because language can authorize external operations. Yet command language begins to fray almost immediately.

Consider "again," "like before," "this," or "that one." These expressions can direct consequential action while carrying remarkably little of the information needed to determine the action. Their economy is not a defect. It is evidence that the work has moved elsewhere.

Suchman's account of situated action makes that elsewhere difficult to treat as optional decoration. She argues that intelligibility is achieved on particular occasions through recourse to situation particulars, rather than secured once and for all by the linguistic expression itself (Suchman 1985). Clark and Wilkes-Gibbs make a related move in their account of reference: referring is not exhausted by a speaker producing a sufficiently informative noun phrase. It is an iterative achievement in which an expression can be accepted, repaired, expanded, or replaced (Clark and Wilkes-Gibbs 1986).

The prompt "that one" therefore poses a small but destructive question. Where is the command? In the two words? In the previous exchange that made one object salient? In the selection state of the interface? In the model's binding of the demonstrative? In the user's willingness to accept the action that follows?

The obvious rescue is to say that the command consists of the words *plus context*. But "context" has already begun to do suspiciously much work. It names everything excluded by the first boundary without telling us whether those exclusions are of the same kind.

Austin's speech-act analysis adds another complication. Utterances can do things, but not because certain strings possess action as an intrinsic semantic property. Their force depends on procedures, circumstances, roles, and conditions of felicity (Austin 1962). The language may be short because the institution around it is long.

A prompt can therefore behave like a command without being a self-contained command. The very cases that make prompting feel powerful—a tiny phrase steering a large operation—are often the cases in which the visible phrase explains least by itself.

3 Description does not wake up

Another seductive picture begins with description. Describe a room and an image appears. Describe an interface and code arrives. Describe an action and an agent performs it. The old sentence seems to have acquired new consequences.

But nothing woke up inside description.

Winograd's SHRDLU makes this easier to see because the apparatus is exposed rather than hidden. English utterances were interpreted against a deliberately constructed blocks world containing entities, relations, procedures, and state; interpreted sentences could then be converted into executable instructions (Winograd 1973). The consequentiality of the language depended on the machinery that made particular interpretations and actions possible.

The historical example matters because it blocks a cheap story of novelty. Natural language did not first become computationally consequential with large language models. What changes across systems is where the restriction and interpretive burden are placed. SHRDLU bought expressive freedom by making the world small. Attempto bought reliable execution by restricting the language itself (Fuchs and Schwitter 1996). Contemporary systems relax some of those visible restrictions, but the need for an actionable arrangement has not disappeared. It has become harder to see.

The sentence "make the house remember" can therefore belong to a poem in one setting and become an instruction in another without changing a word. The shift is real, but it is not located inside the sentence. A system has been built in which the sentence enters a chain of interpretation, generation, tool use, and state change.

This is where the phrase "description becomes operation" becomes dangerous. It compresses the apparatus into the grammar. It attributes the consequence to the sentence because the infrastructure has succeeded in disappearing.

Yet the opposite slogan—"the apparatus does everything"—would be just as empty. Different descriptions produce different trajectories through the same apparatus. Wording matters. Examples matter. Sequence matters. The point is not to evacuate agency from language but to stop granting language the agency of the entire arrangement.

The prompt slips here because the interface offers one visible cause while the operation is produced by a relation.

4 Program, before and after the text

Programming seems to offer a firmer analogy. A prompt specifies behavior; the system executes it. Current systems make that resemblance unusually literal. Natural-language instructions can constrain structured output, select tools, and even cause a model to synthesize temporary code that orchestrates several operations (OpenAI 2026b; OpenAI 2026a).

But the analogy becomes stranger the closer it gets.

If the prompt is the program, what is the generated JavaScript that actually loops, branches, and calls tools? If that generated code is the program, what are the tool schemas that determine which calls are legal? If the runtime assemblage is the program, why call the user's words a program at all?

Older work refuses the simple chronology in which prompts are fuzzy and code is exact. Pygmalion made concrete examples part of program construction; programming by demonstration treated visible action as a way of specifying behavior before a complete symbolic account existed (Smith 1993). SHRDLU had already translated ordinary-language interaction into procedures. Attempto deliberately engineered a controlled natural language that could be mapped into formal representations. The border between words and programs has been repeatedly crossed by changing what surrounds the words.

Naur cuts deeper because he attacks the comparison from inside programming. In "Programming as Theory Building," program text and documentation do not contain the whole competence required to understand, explain, and modify a program. The programmer carries a theory of the matters

handled by the program that exceeds the artifact (Naur 1985). Programming is not simply the production of a fully explicit text.

That does not make prompts and programs identical. Formal programming languages still provide syntactic and semantic constraints that ordinary-language prompting often lacks. But it destroys a convenient contrast: code cannot stand for complete specification while prompting stands for incomplete intention. Real programming was never so clean.

The prompt disappears again. Call the user utterance the program and executable control flow appears downstream. Call the executable control flow the program and the upstream judgment that produced and revised it falls out. Call the whole arrangement the program and the word has expanded until it risks becoming a synonym for system.

5 The specification that arrives late

Perhaps "specification" avoids these problems. A prompt need not itself execute; it can state requirements, constraints, examples, or desired outcomes. This seems closer to what people actually do.

Then the first failure arrives.

The user says no. Something was too glossy, too literal, too symmetrical, too fast, too safe. A new sentence is added. Another output appears. The user says no again, differently. After several turns, the instruction that would have produced the accepted result becomes clearer than it was at the beginning.

Suchman's analysis of breakdown helps explain why the failure matters. Trouble is not simply absence of success. An inappropriate response forces a diagnosis: was the utterance misheard, misunderstood, attached to the wrong referent, or interpreted under the wrong assumptions? Repair follows the diagnosis (Suchman 1985). The failed response can reveal a hidden interpretation that the original specification never made visible.

This does not mean every failure is epistemically rich. A stochastic system can produce garbage without exposing a stable misunderstanding. Nor does every success demonstrate correct interpretation; lucky output can conceal the fact that the system got there for reasons the user would reject. But failure creates a peculiar possibility: the artifact talks back to the specification.

Programming by demonstration again provides an older precedent. Examples can function as partial specifications whose implications are discovered through use rather than completely stated in advance (Smith 1993). Contemporary prompt iteration intensifies the pattern because generation is cheap enough for provisional artifacts to become routine instruments of thought.

The harder case is tacit judgment. An operator may reject ten outputs and only later find language for what was wrong. The criterion was not absent; it governed selection. Yet it was not available in propositional form when the process began. "Specification" then names something distributed across the initial instruction, the generated candidates, the acts of rejection, and the eventual articulation of a distinction that had first appeared only as discomfort.

At that point saying "the prompt is the specification" becomes temporally confused. Which prompt? The first one, which did not know enough? The last one, which records what the failures taught?

The entire genealogy? The accepted artifact itself?

The specification is not merely written and then executed. In these cases, part of it is excavated from the consequences of trying to act without it.

6 Context is where distinctions go to hide

The easiest way to rescue every failed definition is to add context. The prompt is the command in context, the description in context, the program in context, the specification in context.

But context is not one thing.

Winograd's system maintained an explicit model of a restricted world and discourse. Suchman's situation particulars concern the occasion through which intelligibility is achieved. Brooks, attacking centralized representation, argues for ongoing perception-action coupling with the environment rather than treating intelligence as manipulation of an internal world model (Brooks 1991). These are not three versions of the same container.

A contemporary system can also carry several different things under the word at once: previous messages, retrieved documents, model weights, persistent memory, a selected image region, a file tree, a browser state, tool outputs, permissions, a current location, or a live sensor stream. Some are representations. Some are capabilities. Some are histories. Some are relations to a world that can change while the system is acting.

The distinction matters because "put more context in the prompt" treats access as if it were equivalent to description. A model given a paragraph describing a changing environment does not have the same relation to that environment as an agent that can look again after acting. More representation is not the same as renewed coupling.

Nor is a representation of someone's situation equivalent to being situated as that person. A user can describe where they are, what they remember, what embarrasses them, what happened yesterday, what kind of sentence feels false. The description may improve the model's response. It does not follow that the supplied text has exhausted the position from which the judgment is made.

"Context" becomes dangerous precisely because it is useful. It absorbs whatever the current explanation forgot. Once it does, the missing distinctions disappear inside the repair.

The prompt is no easier to locate after the rescue. It has simply been surrounded by a word large enough to hide the crime scene.

7 The receipt does not contain the author

One response to all this instability is to stop arguing about invisible contribution and preserve the trace. Keep the prompts, outputs, rejected candidates, edits, branches, selections, and final arrangement. Instead of asking who wrote the work from memory or etiquette, inspect the production history.

This is an important improvement. It is not a verdict.

A complete trace can show that one person originated the project, another system generated sentences, the person rejected most of them, a model reorganized a section, the person rewrote the ending, and the final artifact was published under one name. Nothing in the trace itself decides which of these operations should control the word *authorship*.

The temptation is to replace a metaphysical author with a distributed process. Hutchins gives a serious account of distributed cognition in which representational states propagate and transform across people and artifacts (Hutchins 1995). The crucial word is not *distributed*; it is *transform*. Distribution becomes explanatory when we can say what changed at each passage.

The same discipline is needed for writing. Generation, selection, rejection, revision, arrangement, endorsement, and responsibility are not interchangeable contributions. A sentence may be generated by one component, made necessary by another, selected by a third operation, and made consequential by the person who chooses to publish it. Causal influence can spread while responsibility remains concentrated.

The trace therefore does something both smaller and more useful than solving authorship. It removes excuses for pretending the production process was simpler than it was. After that, the difficult question still remains.

This is another disappearance. Search the receipt for the author and find operations. Gather the operations into "distributed authorship" and discover that the distribution itself does not tell us how praise, blame, ownership, intention, or responsibility should be assigned.

8 A noun with a moving border

The failures now begin to resemble one another, but not enough to become a grand unified theory.

Command fails because reference can live outside the utterance. Description fails because consequence lives in an apparatus. Program fails because executable text is only one layer of programming practice. Specification fails because the task can change through the attempt to satisfy it. Context fails because unlike mechanisms are hidden under one word. Provenance fails because a causal record does not carry its own attribution rule. Distribution fails when multiplicity is named before transformation is described.

There is a recurrent movement here. Draw a tight boundary around the visible utterance and important causes fall outside it. Expand the boundary to include those causes and the object called "the prompt" becomes harder to distinguish from the interaction or system in which it participates.

That movement may be more informative than another definition.

It suggests that "prompt" is stable as an interface noun before it is stable as an explanatory object. Interfaces need handles. Research needs cuts. The handle offered by the interface is not automatically the cut demanded by the question.

If the question concerns lexical sensitivity, the visible utterance may be exactly the right object. If the question concerns reference, prior interaction and selection state may have to enter. If the question concerns tool use, permissions and schemas become part of the causal account. If the question concerns authorship, post-generation judgment and revision cannot be treated as aftermath. Different questions force different borders.

The mistake is not choosing a border. Explanation is impossible without one. The mistake is forgetting that the border was chosen and then attributing to the thing inside it the work done by what was cut away.

This is why the prompt keeps disappearing. Not because nothing is there. Something was typed. Something happened. The disappearance occurs when a practical object is asked to carry an explanation larger than its boundary.

9 What the failure opens

The instability changes several research questions that otherwise look settled too quickly.

The question "are prompts programs?" becomes less useful than asking which operations are formalized where, and which remain dependent on interpretation. "Does the prompt contain the specification?" becomes a question about when criteria become articulate and whether failure changes the task rather than merely improving compliance. "Does more context improve understanding?" fractures into questions about represented state, interaction history, renewed access to a changing environment, and the user's situated judgment. "Who wrote it?" becomes impossible to answer responsibly until generation, selection, revision, endorsement, and responsibility stop being treated as one operation.

None of these replacements guarantees a final theory. They do something more modest: they make it harder for one convenient noun to settle disputes that belong to different mechanisms.

The historical record should make us suspicious of declarations of firstness as well. Executable natural language, programming by demonstration, controlled language, situated action, and theories of programming that exceed program text all precede large language models (Winograd 1973; Smith 1993; Fuchs and Schwitter 1996; Suchman 1985; Naur 1985). Contemporary systems may recombine these problems at unusual scale, with unusual fluency and unusually porous boundaries between language, generation, tools, and external action. That is consequential. It is not the same as saying the underlying problems were waiting for 2022 to exist.

The difficult historical question is therefore not whether everything is old or everything is new. It is where earlier systems had to make their restrictions explicit and where contemporary systems are able to postpone, distribute, or conceal them.

Even that sentence should remain under pressure. "Postpone" may describe one mechanism and distort another. The point is not to replace one master word with a better master word. It is to keep the machinery visible long enough for the differences to survive.

10 The prompt keeps disappearing

A prompt can be a command. It can be a description. It can participate in a program, carry a specification, exploit a history, leave a provenance trail, and occupy one position in a distributed process. None of those statements is especially controversial. Trouble begins when one of them is promoted from a local description into the identity of the object.

Then the counterexample arrives.

The command points outside itself. The description borrows its force from machinery. The program extends beyond the text. The specification learns from its own failures. Context splits into different kinds of state and relation. The receipt refuses to say who the author is. Distribution becomes fog until the transformations are named.

The prompt survives ordinary use perfectly well. We can keep calling the thing in the box a prompt. The scholarly problem begins when the box is mistaken for the boundary of the phenomenon.

What follows is not that the boundary should expand without limit. That would only produce a new abstraction large enough to swallow every objection. The more difficult practice is to cut narrowly, name what the cut excludes, and be willing to move it when the explanation breaks.

Perhaps a stable theoretical object will eventually survive that pressure. At present the more interesting fact is that the object keeps changing shape under examination.

The prompt is easy to point at.

It is harder to say where it ends.

References

Austin, J. L. (1962). *How to Do Things with Words*. Ed. by J. O. Urmson. Oxford: Clarendon Press.

Brooks, Rodney A. (1991). "Intelligence without Representation". In: *Artificial Intelligence* 47.1–3, pp. 139–159.

Clark, Herbert H. and Deanna Wilkes-Gibbs (1986). "Referring as a Collaborative Process". In: *Cognition* 22.1, pp. 1–39. DOI: 10.1016/0010-0277(86)90010-7.

Fuchs, Norbert E. and Rolf Schwitter (1996). *Attempto Controlled English (ACE)*. arXiv: cmp-lg/9603003. URL: https://arxiv.org/abs/cmp-lg/9603003.

Hutchins, Edwin (1995). *Cognition in the Wild*. Cambridge, MA: MIT Press.

Naur, Peter (1985). "Programming as Theory Building". In: *Microprocessing and Microprogramming* 15, pp. 253–261.

OpenAI (2026a). *Programmatic Tool Calling*. API documentation; accessed 2026-08-17. URL: https://developers.openai.com/api/docs/guides/tools-programmatic-tool-calling.

— (2026b). *Structured Model Outputs*. API documentation; accessed 2026-08-17. URL: https://developers.openai.com/api/docs/guides/structured-outputs.

Smith, David Canfield (1993). "Pygmalion: An Executable Electronic Blackboard". In: *Watch What I Do: Programming by Demonstration*. Ed. by Allen Cypher. MIT Press. URL: https://acypher.com/wwid/Chapters/01Pygmalion.html.

Suchman, Lucy A. (Feb. 1985). *Plans and Situated Actions: The Problem of Human-Machine Communication*. Tech. rep. ISL-6. Palo Alto, CA: Xerox Palo Alto Research Center.

Winograd, Terry (1973). "A Procedural Model of Language Understanding". In: *Computer Models of Thought and Language*. Ed. by Roger C. Schank and Kenneth M. Colby. San Francisco: W. H. Freeman, pp. 152–186.